

House Price Predictor — Explained From Zero

A complete beginner's guide to your own project + interview questions with answers

Project: California House Price Predictor **Live demo:** <https://sandeep-house-price.streamlit.app> **Code:** <https://github.com/Sandeon-8104/house-price-predictor>

PART 1 — WHAT IS MACHINE LEARNING? (start here)

1.1 The one-line idea

Normal programming: **you write the rules**, the computer follows them.

```
IF income is high AND near the beach THEN price is high
```

Machine learning: **you show the computer thousands of examples**, and it figures out the rules **by itself**.

That's it. That's machine learning. Everything else is detail.

1.2 A child's analogy

Imagine a kid who has never seen fruit. You show them 1,000 fruits and say "apple" or "orange" each time. After a while, the kid learns by themselves: "*round + red = probably apple, round + orange color = probably orange.*" Nobody gave the kid rules — the kid **learned patterns from examples**.

Our project does exactly this, but instead of fruits it looks at **districts of California**, and instead of saying "apple/orange" it says "**this district's houses cost about \$250,000.**"

1.3 The two words you must know

- **Features** = the clues we give the model (income of the area, number of rooms, location...). Think: *the inputs*.
- **Target** = the answer we want it to guess (house price). Think: *the output*.

Training = showing the model thousands of (clues → answer) pairs until it learns the connection.

1.4 What "model" means

A **model** is just a file with learned patterns inside. Before training it knows nothing. After training, it's like the kid who has seen 1,000 fruits — you show it a NEW district it has never seen, and it guesses the price.

1.5 Regression vs Classification (one important word)

- **Classification** = guessing a category ("apple or orange?", "spam or not?")
- **Regression** = guessing a **number** ("price = \$253,000")

Our project predicts a number → it is a **regression** project.

PART 2 — OUR PROJECT, STEP BY STEP, LIKE A STORY

Step 0 — Setting up the kitchen 🔍

Before cooking you need a kitchen and tools. Our tools:

Tool	What it does (simple words)
Python	The language we speak to the computer
pandas	Excel inside code — holds our table of data
matplotlib / seaborn	Draws charts so we can SEE the data
scikit-learn	The ML toolbox — models are pre-built, we just use them
joblib	Saves the trained model into a file
Streamlit	Turns Python code into a website with sliders

We installed all of them with one command: `pip install ...`. No special computer needed — a normal laptop is enough because our data is small.

Step 1 — Getting the data 📦

You can't learn from examples without examples! We used the **California Housing dataset** — real data from the 1990 US census.

- **20,640 rows** — each row is one small district of California
- Each row says: how rich people there are, how old houses are, how many rooms, where it is on the map, AND the actual house price

Think of it as a giant Excel sheet: 20,640 lines, 9 columns. The last column (price) is the **target**; the other 8 are **features**.

Why is this dataset famous? It's the "hello world" of regression — clean enough for beginners, messy enough to teach real lessons.

Step 2 — Looking at the data first (EDA) 🔍

EDA = Exploratory Data Analysis = "look before you cook."

You wouldn't cook with ingredients without checking if they're rotten. Same with data. We printed statistics and drew charts. What we found:

Finding 1 — Income is king. We computed **correlation** — a number between -1 and 1 that says how strongly two things move together:

- +1 = they rise together perfectly
- 0 = no connection at all
- -1 = one rises exactly when the other falls

Income vs price = **0.688** → strong! Rich areas have expensive houses. No surprise, but now it's *proven* with a number.

Finding 2 — The map doesn't lie. We plotted every district on a map, colored by price. Result: red (expensive) clusters exactly around **San Francisco and Los Angeles**. But here's the twist: latitude alone had correlation of only -0.144! Why? Because correlation only sees *straight-line* relationships, and location works in a curvy, complex way ("near a city from ANY direction = expensive"). Remember this twist — it becomes important in Step 4.

Finding 3 — The data has problems: - Prices stop at exactly **\$500,000** — the census people **capped** them. A \$2 million district was written down as \$500k. That's a lie in the data! - Impossible rows: a district with **141 average rooms** per house, another with **1,243 people per household**. These are probably prisons, dorms, or typing mistakes.

Step 3 — Cleaning the data 🧹

Now we fix the problems we found:

1. **Deleted 992 capped rows.** Why not keep them? Because the model would learn "no house is ever above \$500k" — a false rule. Better to have no answer than a wrong answer.
2. **Deleted 162 crazy-outlier rows** (the 141-room "houses" etc.). These are not normal homes; they would confuse the model.

Left: **19,486 rows** — we lost only 5.6% and everything left is trustworthy.

Golden rule we followed: never overwrite the original file. We saved a NEW file (`clean.csv`). If we regret a cleaning decision, the original is safe.

Step 4 — Feature engineering (the magic step) ✨

This is the step where OUR brain helps the model's brain.

The model sees Latitude = 37.77, Longitude = -122.42 and thinks "just two numbers, whatever." But WE know those numbers mean **San Francisco**! The model can't easily figure out "near a big city = expensive" from raw coordinates. So we computed it FOR the model:

```
DistToCity = distance to the nearest big city (SF or LA)
```

We also created: - **RoomsPerPerson** = rooms ÷ people → spacious homes = richer area - **BedrmsRatio** = bedrooms ÷ rooms → fancy houses have big living rooms and offices, so a LOWER ratio often means a FANCIER house

The result was beautiful: DistToCity has correlation **-0.452** with price (negative = farther from city → cheaper). We took two nearly useless columns and built the **2nd strongest clue in the whole dataset**. This is called **feature engineering** — encoding human knowledge as new columns. It often improves models more than switching to a fancier algorithm.

Step 5 — The exam rule: train/test split 📄

Here's a school analogy. If the teacher gives you the exam questions AND answers to memorize, then tests you on the SAME questions — a perfect score means nothing. You memorized; you didn't learn.

Same for models. So we split our data:

- **80% (15,588 rows) = training set** → the model studies these
- **20% (3,898 rows) = test set** → hidden in a drawer, model NEVER sees them

At the end, we grade the model only on the hidden 20%. If it does well on data it has never seen, it truly learned the patterns — it didn't memorize.

Step 6 — Training four models (the competition) 🎲

"Training" sounds fancy but in code it's ONE line: `model.fit(X_train, y_train)`. The model looks at all the training rows and adjusts itself to fit the patterns. We trained four contestants, from simplest to fanciest:

Contestant 1 — Linear Regression (the ruler 📏). Draws the best straight line through the data. Like saying "every extra \$10k income adds about \$40k to price." Simple, fast, honest. Score: $R^2 = 0.654$.

Contestant 2 — Decision Tree (the 20-questions game 🎮). Learns a flowchart: "Income > \$50k? → yes → Near coast? → yes → expensive!" Sounds smart, BUT it scored **0.620 — WORSE than the straight line!** Why? It memorized the training data (like the student who memorizes answers) and choked on the hidden test data. This failure has a name: **overfitting**. Our project demonstrated it live.

Contestant 3 — Random Forest (the crowd of 100 🌲 🌲 🌲). Instead of one tree, grow **100 trees**, each seeing a random slice of the data, and let them **vote**. One tree makes silly mistakes, but 100 different trees make DIFFERENT silly mistakes — and the average of their answers is right. Like asking 100 people to guess the weight of a cow: individuals are wrong, the average is amazingly accurate. Score: **$R^2 = 0.805$** — **WINNER 🏆**

Contestant 4 — Gradient Boosting (the mistake-fixer chain 🧑🔧). Trees built one after another, where each new tree focuses on fixing the errors of the previous ones. Score: **0.792** — close second.

Model	R^2	Typical error
Linear Regression	0.654	\$42,825
Decision Tree	0.620	\$40,013
Random Forest 🏆	0.805	\$28,691
Gradient Boosting	0.792	\$31,107

Total training time for all four: about 13 seconds on a normal laptop.

Step 7 — Grading the winner properly 🎓

What is R^2 (R-squared)? Imagine you must guess every house price but you know NOTHING — your best strategy is to guess the average price every time. R^2 measures how much better than that dumb strategy your model is:

- $R^2 = 0$ → no better than always guessing the average
- $R^2 = 1$ → perfect predictions, every time
- **Our 0.805 → the model explains 80.5% of why prices differ**

What is the error in dollars? On average our predictions are off by about **\$29,000**. Typical district price is ~\$190,000, so roughly 15% off. For 13 seconds of training on free data — quite good!

Which clues did the model actually use? We can ask it! Feature importance:

1. **MedInc (income): 0.416** — by far the biggest clue (as EDA predicted!)
2. **DistToCity: 0.143** — OUR ENGINEERED FEATURE at rank #2! 🇺🇸
3. AveOccup: 0.118, then everything else small.

The plot of predicted vs actual prices shows dots hugging the "perfect" line — tight for cheap districts, more scattered for expensive ones (expensive houses are simply harder to predict; their prices depend on things not in our data, like sea views).

Step 8 — Saving the model 💾

Training happens once; using the model happens forever. We saved the trained forest into a file:

```
joblib.dump(model, "house_price_model.pkl", compress=3)
```

The file was 135 MB (100 trees are heavy!). GitHub refuses files over 100 MB, so we compressed it to **31 MB** — like zipping a folder. Loading it back is one line, and predictions are instant. No retraining ever needed.

Step 9 — The web app (making it real)

A model in a file is invisible. We built a **web page** around it using Streamlit: you move sliders (income, rooms, location...), and the model predicts the price **live**, shown in big numbers with a map.

Then we deployed it to **Streamlit Community Cloud** (free): it takes our code straight from GitHub and hosts it at a public link — **<https://sandeep-house-price.streamlit.app>** — anyone in the world can open it. Try it: set location to San Francisco with high income → ~\$400k. Switch to inland Fresno with low income → ~\$94k. The model clearly learned that income + location drive prices.

The whole story in five lines

1. **Got** 20,640 real examples of district → price.
2. **Looked** at the data, found lies (capped prices) and garbage (141-room houses); **cleaned** them out.
3. **Added smarter clues** (distance to city) using human knowledge.
4. **Trained** four models on 80% of data, **graded** them fairly on the hidden 20% — Random Forest won ($R^2 = 0.805$, ~\$29k error).
5. **Saved** the winner and **shipped** it as a public website.

That is the complete machine learning workflow. Every real ML project — at Google, at a bank, anywhere — follows this same skeleton, just with bigger data and more people.

PART 3 — INTERVIEW QUESTIONS & ANSWERS

How to use this section: don't memorize word-by-word. Read the answer, close your eyes, say it in your own words. The stories from YOUR project (Decision Tree failing, DistToCity ranking #2) are your strongest weapons — real experiences beat textbook definitions.

A. Project overview questions

Q1. Walk me through your project.

A: "I built an end-to-end house price predictor on the California Housing dataset — 20,640 districts from the census. I did EDA and found two data problems: prices capped at \$500k and impossible outliers, which I removed. I engineered new features — the most successful was distance-to-nearest-city, which I computed from raw coordinates; it became the model's 2nd most important feature. I compared four

models with an 80/20 train-test split: linear regression as baseline, then a decision tree — which actually overfit and scored worse than linear — then Random Forest and Gradient Boosting. Random Forest won with R^2 of 0.805, a typical error around \$29,000. I saved the compressed model and deployed it as a public Streamlit web app where anyone can move sliders and get a live prediction."

(Practice this until it's smooth — it's 45 seconds and covers everything.)

Q2. Why did you choose this dataset/project?

A: "It's the classic regression project, and that's exactly why — it teaches the complete workflow: EDA, cleaning decisions, feature engineering, model comparison, and deployment. The dataset is small enough to iterate fast but has real problems — capped values, outliers — so the cleaning decisions are real, not artificial."

Q3. Is this regression or classification, and why?

A: "Regression — the target is a continuous number (median house value). Classification would be predicting a category, like 'expensive vs cheap.' If the business only needed price *buckets*, I could reframe it as classification, but a dollar number is more useful here."

B. Data questions

Q4. How did you handle missing values?

A: "This dataset had none — I verified with `isnull().sum()`. But my pipeline still includes a median-fill fallback, because checking is a habit, not an assumption. If there had been missing values, I'd fill numeric columns with the median — it's robust to outliers, unlike the mean — or drop rows if very few were affected."

Q5. Why did you DELETE the capped rows instead of keeping them?

A: "The cap means a district recorded as \$500k might really be worth \$2M — the true value is unknown. If I keep those rows, I'm training the model on wrong answers, and it learns to never predict above \$500k. Deleting 992 rows (under 5% of data) removes the lie entirely. The trade-off is my model can't predict ultra-expensive districts — I'd state that as a known limitation."

Q6. How did you find and handle outliers?

A: "Through `describe()` — the max values were absurd: 141 average rooms, 1,243 average occupants. Those are likely prisons, dorms, or data errors, not homes. I removed rows beyond sensible thresholds — 162 rows total. Important: I decided the thresholds by looking at the distributions, not blindly. And I never touched the raw file — each cleaning stage writes a new CSV, so every decision is reversible."

Q7. What is correlation? What did it tell you?

A: "A number from -1 to +1 measuring how strongly two variables move together linearly. Income correlated 0.688 with price — the strongest single clue. But there's a trap: latitude showed only -0.144, yet the map clearly showed location matters hugely. Correlation only captures straight-line relationships;

location affects price in a non-linear way. That insight drove both my feature engineering and my choice to try tree-based models."

C. Feature engineering questions

Q8. What is feature engineering? What did you do?

A: "Creating new input columns that make patterns easier for the model to see — encoding human knowledge as data. I created five: rooms-per-person, bedroom-ratio, and three distance features. The best was DistToCity — distance to the nearest of SF or LA, computed from lat/long. Raw coordinates had near-zero correlation with price; DistToCity reached -0.452 and became the 2nd most important feature in the final model, ahead of almost every original column. It's my favorite result in the project: two useless columns combined into the second-best predictor."

Q9. Why couldn't the model figure out location by itself?

A: "Tree models can partially learn it by splitting on lat and long repeatedly — that's why Random Forest still gave them some importance. But 'distance to a point' takes many axis-aligned splits to approximate, and a linear model can't represent it at all. Handing the model the distance directly makes the pattern one split away instead of twenty. Feature engineering reduces how hard the model has to work."

D. Modeling questions

Q10. Why did you split data into train and test? What if you didn't?

A: "To grade the model on data it has never seen — that's the only honest measure of generalization. Without the split, I'd evaluate on memorized data, and my own project shows the danger: the decision tree looks near-perfect on training data but scored R^2 0.620 on the test set — worse than plain linear regression. Without a held-out test set I would have shipped the worst model thinking it was the best."

Q11. What is overfitting? How did you see it and prevent it?

A: "Overfitting is memorizing instead of learning — perfect on training data, bad on new data. Like a student who memorizes past exam answers and fails when the questions change. I saw it live: my unlimited-depth decision tree underperformed linear regression on the test set. The fixes: ensembles (Random Forest averages 100 trees, and averaging cancels out individual memorization), limiting tree depth, and always evaluating on held-out data."

Q12. Explain Random Forest simply. Why did it win?

A: "One decision tree overfits. Random Forest builds 100 trees, each trained on a random sample of rows and features, and averages their predictions. Each tree makes different mistakes, and the mistakes cancel out in the average — wisdom of the crowd. It won because this dataset has non-linear patterns (location!) that linear regression can't capture, while the averaging protects against single-tree overfitting. It jumped from the linear baseline of 0.654 to 0.805."

Q13. What is Gradient Boosting and how is it different from Random Forest?

A: "Both are tree ensembles, but the strategy differs. Random Forest builds trees **in parallel, independently**, and averages them — reduces variance. Boosting builds trees **one after another**, each new tree trained to correct the errors of the previous ones — reduces bias. Boosting often wins with careful tuning; in my project with default-ish settings it came a close second, 0.792 vs 0.805."

Q14. Why start with Linear Regression if you knew it would lose?

A: "As a baseline. Without a baseline, numbers are meaningless — is 0.805 good? Compared to what? The linear 0.654 tells me exactly how much value the complex model adds. Also, if linear had scored 0.80, I'd choose IT — simpler, faster, explainable. You should always earn your complexity."

Q15. What do R^2 and RMSE mean?

A: " R^2 compares my model to the dumbest strategy — always predicting the average price. 0 means no better than that; 1 means perfect. My 0.805 means the model explains 80.5% of price variation. RMSE is the typical prediction error in the target's units — mine translates to roughly \$29-44k depending on the metric (RMSE vs MAE). I like reporting MAE in dollars because a non-technical person understands 'typically off by \$29k' instantly."

E. Deployment & engineering questions

Q16. How did you deploy it? What problems did you hit?

A: "I saved the model with joblib and built a Streamlit app — sliders for inputs, the same feature engineering computed live, prediction shown with a map. Deployed free on Streamlit Community Cloud straight from my GitHub repo. Two real problems I solved: the model file was 135 MB and GitHub blocks files over 100 MB — joblib's compress option shrank it to 31 MB with zero accuracy loss. And I pinned exact library versions in requirements.txt, because a model pickled with one scikit-learn version can fail to load under another."

Q17. In the app, why must you recompute features exactly like in training?

A: "The model expects the exact same 13 inputs, in the same format, as it saw during training. If the user gives lat/long but training used DistToCity, I must compute DistToCity with the identical formula and identical city coordinates. Any mismatch between training-time and serving-time features is called training/serving skew — one of the most common real-world ML bugs."

Q18. How would you retrain or update this model with new data?

A: "My pipeline is five scripts that run end-to-end: get data → clean → engineer features → train → evaluate and save. With new data I'd drop the new CSV in, rerun the pipeline — about a minute — and compare the new test metrics against the old before replacing the deployed model. In production I'd automate that as a scheduled job with metric-based gating."

F. Improvement questions (they LOVE this one)

Q19. How would you improve the model?

A: "In priority order: **First, better features** — travel-time to city instead of straight-line distance, school ratings, crime rates; features usually beat algorithms. **Second, hyperparameter tuning** — GridSearchCV or Optuna over tree depth, number of trees; boosting especially would gain. **Third, cross-validation** instead of a single split for more reliable estimates. **Fourth, try XGBoost or LightGBM** — usually the strongest on tabular data. **Fifth, quantify uncertainty** — predict a range, not just a point; a buyer wants to know '\$250k $\pm$ \$20k'. And honestly: the data is from 1990 — for a real product I'd need current listings data."

Q20. What are the limitations of your project?

A: "Being upfront: the data is from the 1990 census, so prices are historical, not current market values. It predicts district medians, not individual houses. My cleaning removed capped districts, so it can't predict the ultra-luxury segment above \$500k. Distance is straight-line, not travel time. And a single 80/20 split gives a point estimate of performance — cross-validation would be more robust. Knowing the limitations is part of doing ML honestly."

Q21. (Curveball) Your model predicts a house at \$300k but it sells for \$250k. Why?

A: "Several possible reasons: the model's typical error is $\sim$ \$29k, so some individual misses are expected; the specific house may differ from its district's median — my model predicts district-level medians; there are factors not in my features — condition, renovation, view, urgency of sale; and market timing — my training data is a 1990 snapshot. If misses were systematic rather than random, I'd investigate for bias — e.g. checking residuals by region or price band."

Q22. (Conceptual) What's the difference between AI, ML and Deep Learning?

A: "AI is the broad goal — machines doing tasks that need intelligence. ML is one approach to AI — learning patterns from data instead of hand-coded rules; my project is classic ML. Deep learning is a sub-field of ML using multi-layer neural networks — best for images, text and speech. For tabular data like mine, tree ensembles usually match or beat deep learning while being faster and more explainable — which is exactly why I used them."

Final advice for the interview

1. **Own the story, not the buzzwords.** The interviewer has heard "I used Random Forest" a hundred times. What they haven't heard: "my decision tree overfit and lost to linear regression — here's why, and here's how the forest fixed it." Your failures explained well are worth more than your successes listed.
2. **Lead with the DistToCity story.** It shows the rarest skill: thinking about the DATA, not just the algorithm.

3. **Numbers make you credible.** Don't say "it worked well." Say " R^2 0.805, typical error \$29k, versus the 0.654 linear baseline."
4. **Say "I'd check" instead of guessing.** If asked something you don't know: "I haven't tried that, but here's how I would find out." That's a real engineer's answer.
5. **Have the app open on your phone.** "Want to see it live?" wins interviews.